

CSI473 Project Design Report

REMMOGO - Team 04

Field	Complete
Assignment/phase	Phase 1
Approved project title	Remmogo
Approval decision/date	17/08/26
Team number	4
Members and IDs	Rumbidzai Zanele Magweregwe 202208048 Keoikantse Ronald Tumisang 201502162 Kgotla Osiame Mogaetsho 202005511 Omatla Tendai Manyanda 202004949 Leander Van der Walt 202307966
Repository URL	https://github.com/LeeLeeGMOSPhidip/CSI473-Project
Submission tag	phase1-submission
Date	11/09/2026

1. Executive summary

Remmogo addresses the difficulty University of Botswana students experience when finding and managing rental items such as appliances, furniture and electronics. The system provides a student marketplace for listing, searching, requesting, tracking and returning rental items, with communication, reporting and mutual ratings. Payments remain outside the platform; Remmogo records only the Customer confirmation that payment was made and the Owner confirmation that payment was received. The Phase 1 baseline defines the approved scope, measurable requirements, domain responsibilities, traceability, interaction and lifecycle models, and the architecture drivers that will guide Phase 2.

2. Problem evidence, stakeholders and scope

2.1. Problem statement

At the University of Botswana, students often rent items such as fridges, microwaves and other household essentials to one another. The process is largely manual: students advertise through posters, WhatsApp groups or word of mouth. Without a centralised system, it is difficult to track ownership, rental participants and return dates. This wastes time, reduces trust and makes renting stressful for both Owners and Customers.

Remmogo is proposed as a digital marketplace where students can list, request and track rental items. It provides searchable listings, transaction records, availability controls and due-date notifications. These capabilities make student rentals more transparent, efficient and accountable.

2.2 Evidence of the problem

Evidence/source	Date/origin	What it shows
Lost or disputed items.	Past semesters	Students claim items were not returned or ownership was unclear.
Posters and word-of-mouth listings.	Past semesters	Renting is advertised manually, making it hard to reach all students.

2.3 Approval conditions

Remmogo was approved as suitable for development based on the following conditions and characteristics:

- A genuine stakeholder problem - Rremogo addresses a real problem experienced by students who need to rent or lend items within the university community
- Manageable semester scope - The proposed functionality is sufficiently focused to be designed and implemented within the semester
- Meaningful rules/workflow - The system contains meaningful business rules and a defined rental workflow, including item availability, rental requests, payment confirmation, handover and return
- Measurable quality concerns - Rremogo has identifiable quality concerns that can be evaluated through measurable quality scenarios and testing.
- A testable end-to-end vertical slice - Rremogo supports an end-to-end workflow that can be demonstrated and tested from a student's rental request through to the resulting rental record

2.4 Goals and objectives

Item	Team response
Goal	To create a digital marketplace platform that simplifies and organizes student product rentals at the University of Botswana.
Objective 1	Provide a centralized dashboard where students can list and browse available rental items.
Objective 2	Enable secure transaction records to track who rented what and when.
Objective 3	Send reminders and notifications for return date and next payment to reduce disputes
Objective 4	Improve transparency and trust among students by keeping clear digital records.

2.5 Stakeholders

Stakeholder/role	Need	Concern	Evidence/access
Students (Customers)	Easy access to available items to rent and clear rental terms	Items unavailable or lost due to poor tracking.	Complaints in WhatsApp groups.
Students (Owners)	Reliable way to rent out items and track their status.	Fear of losing property or not being paid	Reports of disputes over damaged fridges
Student Union (Administrators)	Oversight of student activities.	No visibility into rental transactions.	Informal disputes brought to student leaders.

2.6 Scope, assumptions and constraints

In scope	Out of scope
Development of a digital marketplace for student rentals.	Financial transactions outside rentals (e.g., purchases)
Features for listing, searching, rental report and tracking items.	
Notifications for due dates and reminders.	

2.7 Assumptions and Constraints

- Users are University of Botswana students with valid UB email addresses.
- Students have access to a device and internet connection.
- Students are willing to use the system according to the defined rental rules.
- Students make payments to one another outside the platform using an agreed local payment method.
- The system only records payment confirmations and does not process or hold funds.
- Students provide accurate information about their listed items.
- Only low-risk personal items are permitted.
- Applicable University of Botswana and residence rules will be verified before implementation.
- Synthetic or anonymised data will be used for development, testing and demonstrations

2.8 Constraints

- Limited budget and development time
- Need for user training and awareness campaigns
- Project timeline tied to semester schedule

3. Glossary

- Customer: A Student who requests and rents an item.
- Owner: A Student who lists an item for rent.
- Administrator: An authorised user who manages account issues, reports and disputes.
- ItemListing: The description of an item offered for rent, including its category, condition, price and available quantity.

- Rental: The record and lifecycle of an agreement between a Customer and an Owner for an ItemListing.
- Reserved: The Rental state after the Owner accepts a request and the requested quantity is reserved.
- Completed: The Rental state reached after both parties confirm the item return.
- Overdue: The Rental state reached when the due date passes without confirmed return.
- ActiveRental: The Rental state reached after the required payment confirmations and item handover are recorded.

4. Functional requirements and use cases

4.1 Functional Requirements

- **FR-01.** The system shall allow a Student to register an account using a valid University of Botswana email address.
- **FR-02.** The system shall allow a registered Student to sign in and sign out.
- **FR-03.** The system shall allow an Owner to upload and list an item for rent.
- **FR-04.** Each ItemListing shall include a name, category, condition, description, rental price and available quantity.
- **FR-05.** The system shall allow Students to search for listed items by name or category.
- **FR-06.** The system shall require every ItemListing to be assigned to one Category.
- **FR-07.** The storefront shall display only ItemListings with available quantity and shall exclude fully reserved or unavailable listings.
- **FR-08.** The system shall record the Customer confirmation that payment was made and the Owner confirmation that payment was received.
- **FR-09.** The system shall allow both the Customer and Owner to confirm that an item has been returned.
- **FR-10.** The system shall allow the Customer and Owner to rate each other after a completed Rental.
- **FR-11.** The system shall update and display the available quantity of each ItemListing.
- **FR-12.** The system shall provide a chat function through which Students can communicate with other Rental participants or an Administrator.
- **FR-13.** The system shall allow a Student to report an account, ItemListing or Rental issue.
- **FR-14.** The system shall allow an Administrator to manage account issues, review reports, resolve disputes and generate administrative reports.

4.2 Actors

- Customer
- Owner
- Administrator

4.3 Core Use Case Flow

Primary Actor: Student (Customer)

Use Case: Request to Rent Item

Preconditions:

Customer has a valid UB account and is signed in.

Item is available in the marketplace.

Main Flow:

1. Customer searches and selects an item.
2. Customer clicks "Rent" and specifies rental period.
3. The system creates a PendingTransaction and notifies the Owner.
4. The Owner reviews and accepts the request, and the system reserves the requested quantity.
5. The Customer confirms that payment was made outside Remmogo.
6. The Owner confirms that payment was received and confirms the item handover.
7. The system activates the Rental, records the due date, updates availability and schedules return reminders.
8. The Customer and Owner receive confirmation notifications.

Postconditions:

The Rental record stores the Customer, Owner, ItemListing, quantity, agreed price, due date and current status.

The Rental is ready for return confirmation and, after completion, mutual rating.

Alternative Flows:

Item already rented → system blocks duplicate booking and notifies Customer.

Payment not confirmed within 48 hours -> the system cancels the pending Rental, releases the reservation and notifies both parties.

Late return → system flags overdue rental and sends notifications.

Cancellation or no-show -> the system records cancellation, releases reserved quantity and restores availability.

Important Rule to Enforce:

Each rental must be tied to a verified student account to prevent disputes and ensure accountability.

State/Result Created:

A rental record is stored in the system, showing item details, renter, owner, and due date.

Failure/Exception to Test:

If the item is already reserved or rented, the system should block duplicate bookings and notify the student that the item is unavailable.

4.4 Acceptance Criteria

Successful Rental Request

Scenario: Customer requests to rent an item

Given that the Customer has a valid UB account, is signed in and selects an available ItemListing, when the Customer specifies the rental period and quantity and submits the request, then the system creates a PendingTransaction, records the Customer, Owner, ItemListing and requested period, and notifies the Owner.

Payment Confirmation and Rental Activation

Scenario: Customer and owner confirm payment for a rental

Given that the Owner has accepted the request and the requested quantity is Reserved, when the Customer confirms payment made, the Owner confirms payment received and the Owner confirms item handover, then the system records both payment confirmations, activates the Rental, records the due date, updates availability and notifies both parties.

Prevent Duplicate Booking of an Unavailable Item

Scenario: Customer attempts to rent an already rented or reserved item

Given the customer is signed in and browsing the catalog, the item should not appear in the availability list.

Handling Payment Not Confirmed

Scenario: Rental remains pending when payment is not yet confirmed

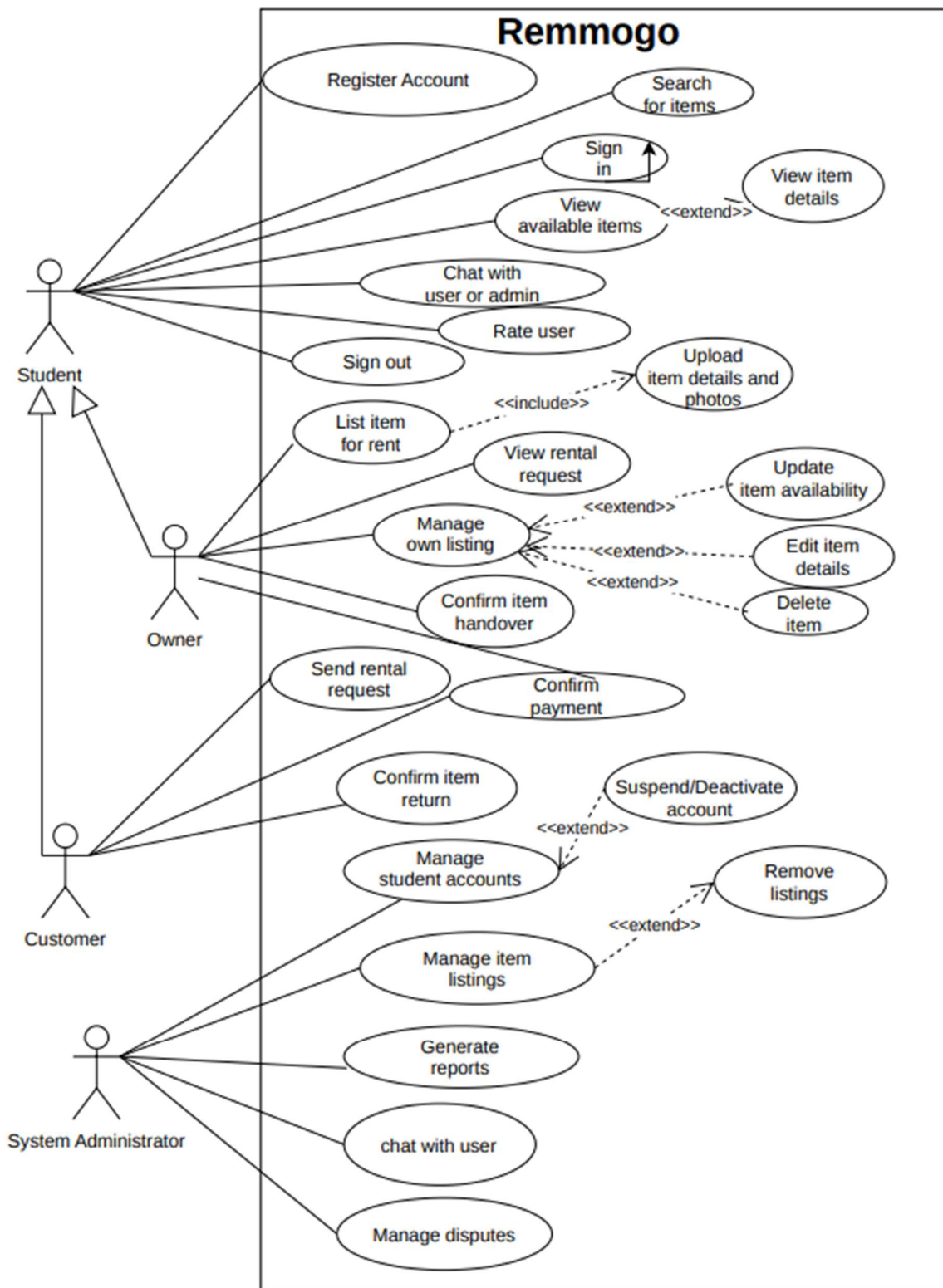
Given that a Rental is waiting for a required payment confirmation, when 48 hours pass without that confirmation, then the system cancels the Rental, prevents activation, releases the reserved quantity and notifies the Customer and Owner.

Return and Overdue Management

Scenario: System handles a late return

Given that an item was rented, reached its due date and the item has not been confirmed as returned when the return deadline passes, then the system will mark the rental as overdue, notify both the customer and owner, and record the overdue status in the rental history.

Use Case diagram



5. Quality requirements

Scenarios

1. Concurrent Login Load

- Stimulus: 50 students attempt to log in simultaneously during peak morning hours.
- Context: Authentication server connected to UB email should support an average daily active user of 500.
- Response: System authenticates all users successfully within 5 seconds.

2. Search Performance for Common Items

- Stimulus: A Customer searches for “fridge”.
- Context: Rremogo catalog has over 1,000 items over 10 categories.
- Response: Search results are displayed within 3 seconds.

3. Transaction Creation Speed

- Stimulus: A rental record is created when a customer confirms payment of a listed item.
- Context: Database holds over 5,000 rental records, which includes item, renter, seller, and due date.
- Response: System stores and confirms the transaction within 2 seconds.

4. Real Time Chat Responsiveness

- Stimulus: A chat message is sent between Owner and Customer.
- Context: Both users are online.
- Response: Message is delivered and visible to recipient in under 3 seconds.

5. Administrative Report Generation

- Stimulus: Administrator generates a report of all rentals for the semester.
- Context: Report includes over 500 records with item details, renter/owner info, and status.
- Response: Report is produced and ready for download within 10 seconds.

6. Overdue Rental Handling

- Stimulus: A rental period passes without return confirmation.
- Context: Transaction due date expired so system checks status at midnight daily.
- Response: System flags the rental as Overdue and sends notifications to both owner and Customer within 1 minute.

Design Obligations

- Use prepared statements for database queries.

- Item state updates throughout the rental process.
- Notification for overdue, return and transaction confirmations.
- A chat service for communication.

These obligations provide measurable targets for authentication, search, transaction creation, chat delivery, report generation and overdue handling.

6. Analysis/domain model and responsibilities

Business rules

- Only users with a valid University of Botswana (UB) email address may register on the system
- Users are assigned roles such as Student (student is either a customer or owner) and Administrator
- Only registered Owners may create item listings
- An item can only be rented if its available quantity is greater than zero.
- A Rental may enter ActiveRental only after the Customer confirms payment made, the Owner confirms payment received and the item handover is recorded.
- All rentals must have a defined return date.
- Only completed rental transactions may be rated.
- Administrators may generate reports on user activity, rentals, and disputes.

Responsibility Allocation

1. Class: Student and Account

- KNOW: UB Email, Password, Name, Role (Customer or Owner).
- DO: Register, sign in/out, manage profile.
- PROTECT: Enforce valid UB email registration and prevent duplicate accounts.
- Collaborate: Administrator for account issues; Rental when participating as a Customer or Owner.

Rationale: Student and Account hold identity and access information so that only valid UB users participate in Remmogo.

2. Class: Owner (Role of User)

- KNOW: OwnerID, owned Items.
- DO: Upload and list items, confirm payment received, confirm item handover and return, and rate Customers.
- PROTECT: Make sure item details are complete before listing and prevent listing unavailable items.
- Collaborate: ItemListing, Rental, Customer, Conversation and Message.

Rationale: Owner owns the knowledge of their items and must enforce listing accuracy.

3. Class: Customer (Role of User)

- KNOW: CustomerID, rental history.
- DO: Search for items, initiate Rentals, confirm payment made, confirm return and rate Owners.
- PROTECT: Make sure rental requests are valid.
- Collaborate: ItemListing, Rental, Owner, Conversation and Message.

Rationale: Customer owns the decision to rent and must confirm transactions.

4. Class: ItemListing

- KNOW: ItemID, Name, Condition, Description, RentalPrice, AvailableQuantity, Category.
- DO: Update availability, provide item details.
- PROTECT: Prevent listing items with quantity below zero or reserved.
- Collaborate: Owner, Category and Rental.

Rationale: ItemListing owns item details and availability information.

5. Class: Category

- KNOW: CategoryID, CategoryName.
- DO: Classify items, allow filtering/search by category.
- PROTECT: Make sure each item belongs to only one category and prevent duplicate category names.
- Collaborate: Item (belongs to category).

Rationale: Category owns classification details to ensure consistency.

6. Class: Rental

- KNOW: TransactionID, StartDate, EndDate, Status, linked Seller, Renter, Item.
- DO: Create rental record, enforce due date, flag overdue, close transaction after return, create notifications.
- PROTECT: Prevent duplicate rentals for the same item, enforce payment confirmation before activation, enforce return confirmation before rating.
- Collaborate: Customer, Owner, ItemListing, PaymentConfirmation, ReturnConfirmation, Rating, Conversation, Message and Administrator.

Rationale: Rental owns lifecycle knowledge and coordinates rental rules, state changes and exceptions.

7. Class: Rating

- KNOW: RatingID, Score, Comment.
- DO: Store feedback, link to transaction.
- PROTECT: Only allow ratings after both parties have confirmed return.
- Collaborate: RentalTransaction (context), Owner/Customer (author and subject).

Rationale: Rating owns evaluation knowledge.

8. Classes: Conversation and Message

- KNOW: MessageID, Timestamp, Content.
- DO: Store and deliver communication.
- PROTECT: Make sure messages are linked to a specific transaction.
- Collaborate: Customer, Owner and Rental.

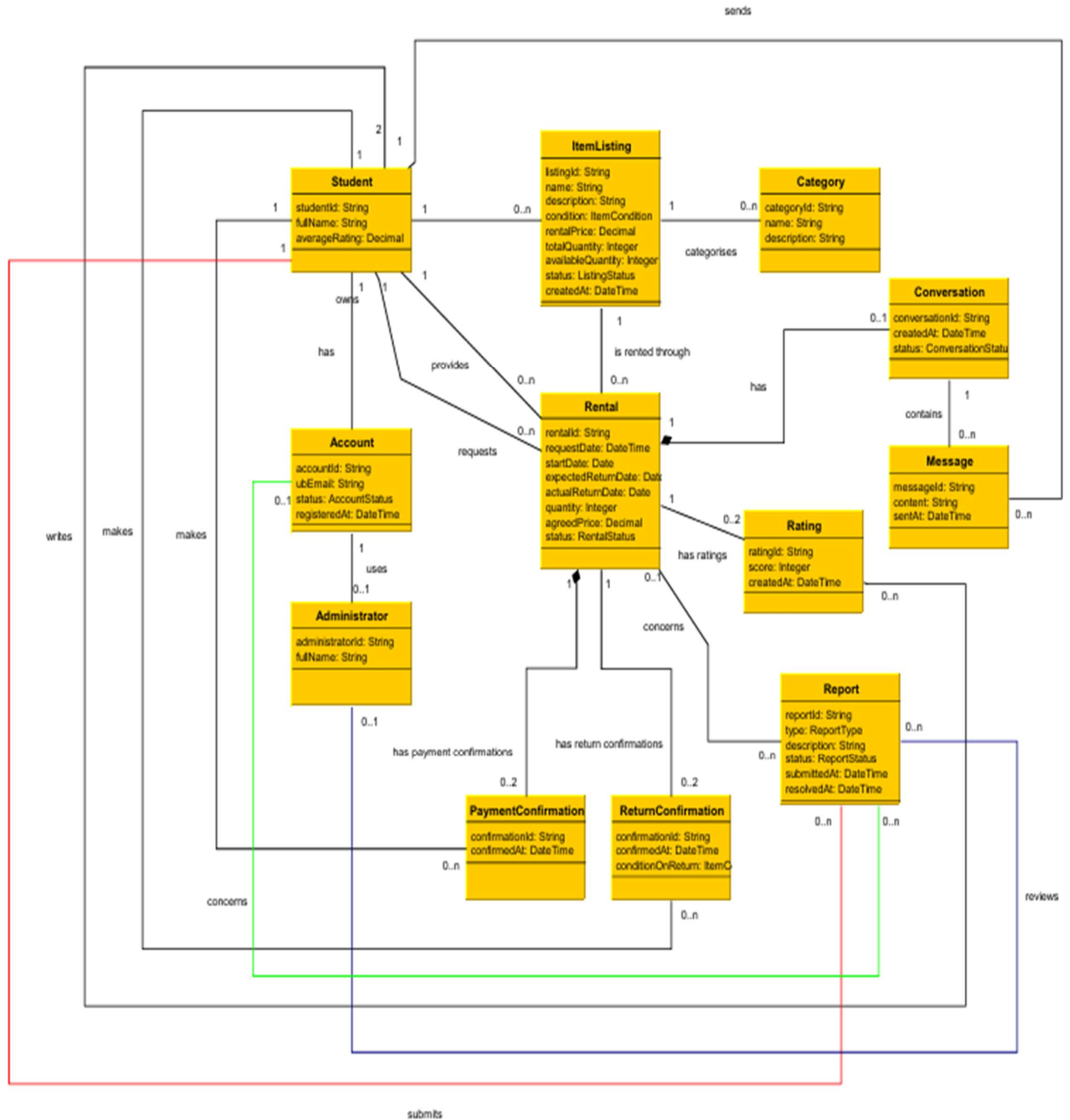
Rationale: ChatMessage owns communication state, ensuring traceability.

9. Classes: Administrator and Report

- KNOW: AdminID, privileges, system policies, dispute resolution rules.
- DO: Manage accounts, resolve disputes (late returns, damaged items, cancellations), generate reports.
- PROTECT: Enforce fairness and compliance with university/residence rules; deactivate accounts when necessary.
- Collaborate: Student, Account, Rental, ItemListing and Report.

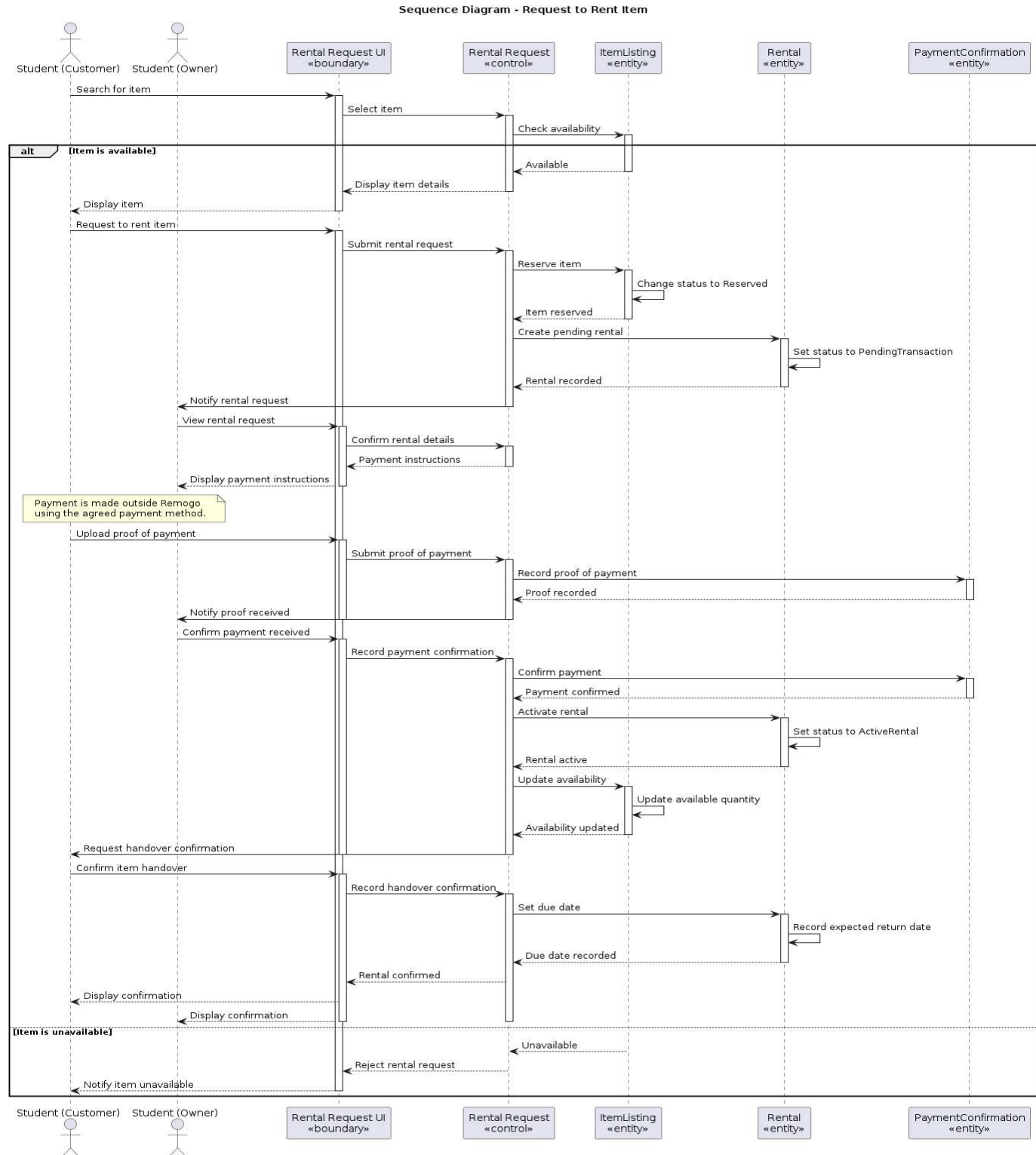
Rationale: Administrator owns oversight responsibilities, ensuring compliance and system integrity.

Domain Model



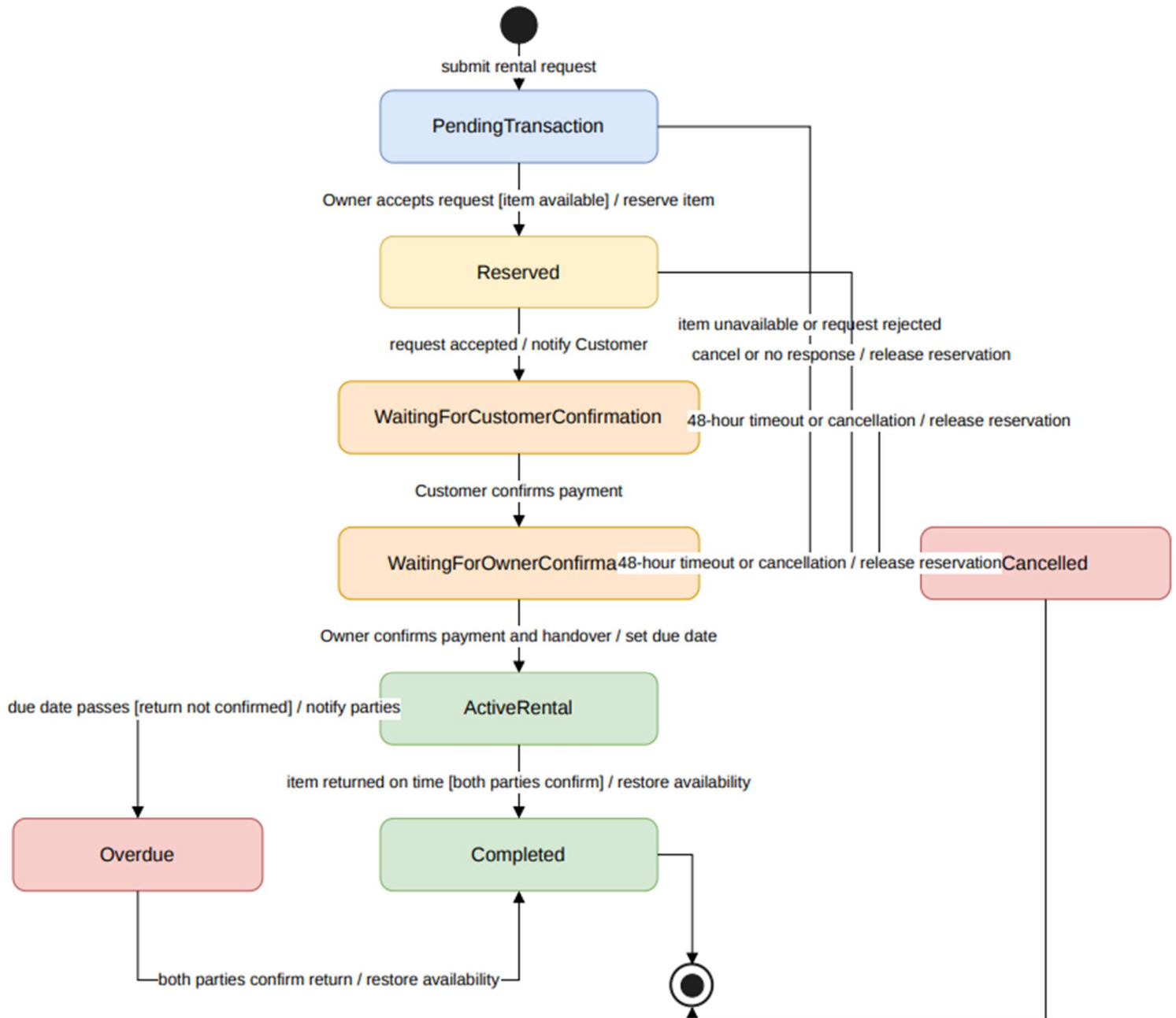
7. Interaction and lifecycle

Sequence Diagram



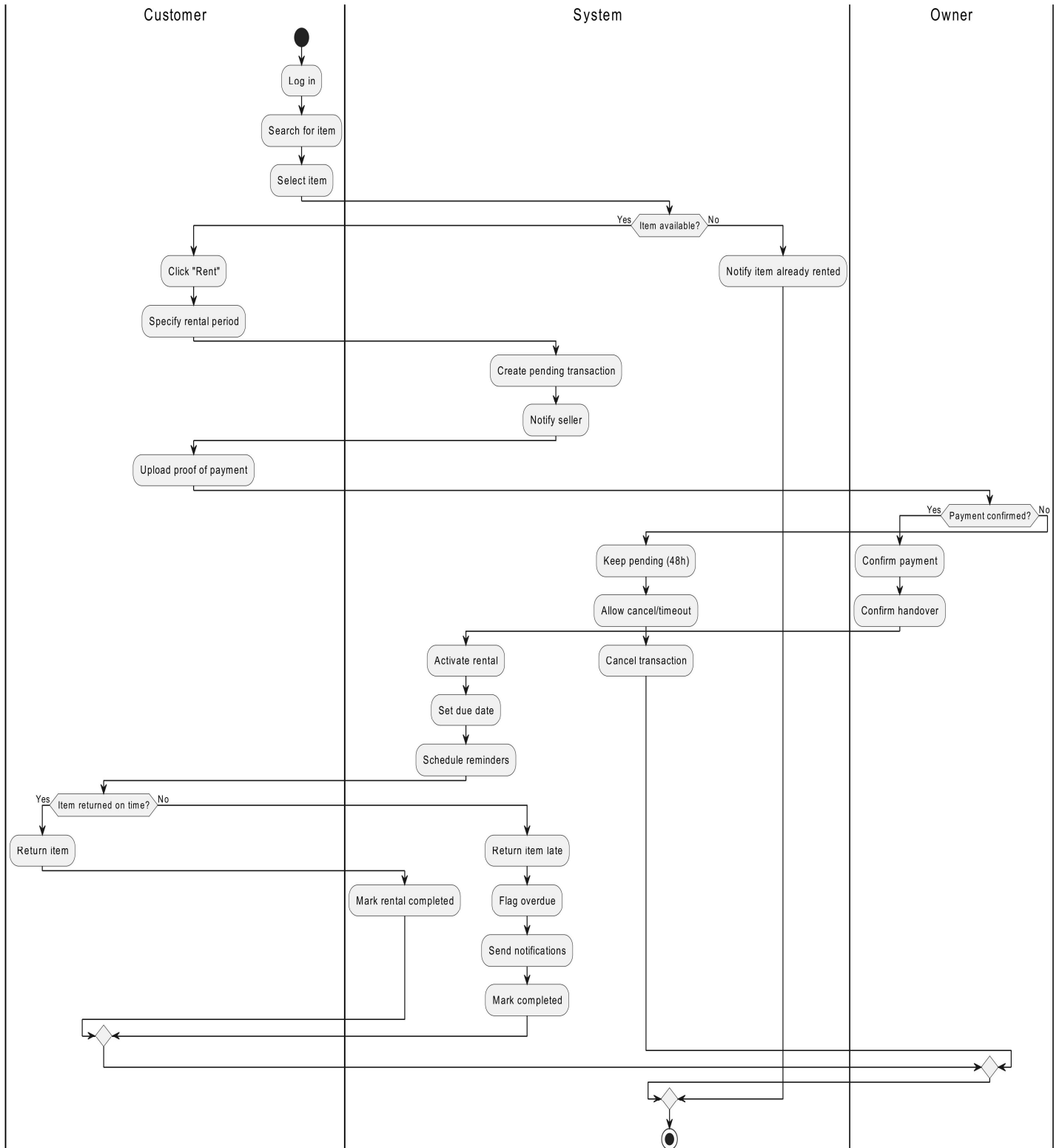
State Machine model

Remmogo Rental State Machine



Activity Model

Activity Diagram - Request to Rent Item



Consistency Explanations

Requirements, Use Cases and Sequence Diagrams

The functional requirements, use cases and interaction model address the same Rental workflow: an authenticated Customer selects an available ItemListing, submits a request, the Owner responds, confirmations are recorded and availability is protected. The consistency matrix provides the detailed mapping between these steps, messages, responsibilities, states and evidence.

Consistent Terminology and naming

Phase 1 uses the domain names Student, Account, ItemListing, Rental, Category, PaymentConfirmation, ReturnConfirmation, Conversation, Message, Rating, Report and Administrator. Customer and Owner are roles played by a Student. The Rental lifecycle terms are PendingTransaction, Reserved, WaitingForCustomerConfirmation, WaitingForOwnerConfirmation, ActiveRental, Overdue, Completed and Cancelled.

Lifecycle and Sequence models Alignment

The lifecycle model records the Rental from request submission through reservation, payment confirmation, handover, activation, overdue handling and completion or cancellation. The consistency matrix identifies the corresponding interaction message and responsibility for each lifecycle effect.

- For example, accepting an available request reserves quantity; the required confirmations and handover permit activation; a missed due date causes Overdue; and two return confirmations permit completion and restoration of availability.

Consistency in Rules and Exceptions

The main exception flows are unavailable quantity, rejection, payment-confirmation timeout, cancellation and late return. Business rules and acceptance criteria specify the same guards and outcomes, including release of reserved quantity after cancellation.

Design Rationale

The Reserved state was retained because it closes the reviewed availability gap. Once quantity is reserved, it is excluded from storefront availability, preventing another Customer from requesting the same unavailable quantity.

Detailed traceability is provided in Appendix A and in docs/consistency-matrix.md in the tagged repository.

8. Architecture and decisions

Drivers

- AD-01 - Availability integrity. Remmogo must prevent conflicting Rentals when several Customers request the same limited quantity. This follows from FR-07, FR-11 and the duplicate-booking acceptance criterion. Availability validation and quantity reservation must

therefore be controlled together, with reserved quantity released after cancellation or completion.

- AD-02 - Trusted student access. Registration and protected actions must be limited to valid UB accounts. This follows from FR-01, FR-02, the approved scope and the concurrent-login quality scenario. Authentication and authorisation must remain separate from Rental logic and must support 50 simultaneous login attempts within five seconds.
- AD-03 - Timely interaction and lifecycle response. Search results, chat messages, Rental creation and overdue notifications have measurable response targets. This follows from Quality Scenarios 2, 3, 4 and 6. The design must separate user interaction, Rental rules, persistence and notifications so that each concern can be tested and changed independently.
-

Alternatives considered

1. Alternative A - In-app payment processing. Remmogo could integrate with a payment provider and process funds directly.
Decision: Rejected for Phase 1 and the initial implementation.
 - Rationale: It would introduce financial-data handling, provider integration, additional failure modes and security obligations beyond the approved scope.
 - Accepted consequence: Students complete payment externally, so Remmogo can record confirmations but cannot independently verify or reverse a payment.
 -
2. Alternative B - Distributed microservices. Authentication, listings, Rentals, chat and reporting could be deployed as separate services.
Decision: Rejected for the semester implementation.
 - Rationale: Independent deployment and distributed coordination would add operational complexity without evidence that the expected project scale requires it.
 - Accepted consequence: A layered modular application is simpler to build and test, but module boundaries must remain clear to avoid tightly coupled code.
3. Alternative C - No reservation state. Listings could remain available until payment and handover are confirmed.
 - Decision: Rejected.
 - Rationale: Competing requests could exceed available quantity and create double bookings.
 - Accepted consequence: Reservation adds lifecycle and timeout logic, but protects availability integrity.

Component Model

The system follows a three-layer architecture.

Presentation layer

- User registration and login
- Item browsing and searching
- Listing rental items
- Chat interface
- Reports and administration screens

Application Layer

- Manage rental transactions
- Handle payment confirmations
- Manage item availability
- Process ratings and reviews
- Handle notifications
- Enforce rental rules

Data Layer

- User information
- Item listings
- Rental records
- Ratings
- Chat history
- Administrative reports

Architecture Decision Records

ADR-01 - External payment confirmation

Decision: Payments are made outside Remmogo. The system records the Customer confirmation that payment was made and the Owner confirmation that payment was received.

Consequence:

Remmogo stores no card, bank or mobile-money credentials. The platform relies on both participants to provide truthful confirmations.

ADR-02 - UB email verification

Decision: Only valid University of Botswana email addresses may be used for Student registration.

Consequence:

The restricted user base improves accountability, but excludes potential users outside the UB community.

ADR-03 - Built-in chat

Decision: Remmogo provides a Conversation and Message function for communication between authorised Rental participants and Administrators.

Consequence:

Communication remains linked to the relevant Rental, but the chat function adds delivery, access-control and data-retention responsibilities.

Consequences of Design Decisions

Positives

- Simple system architecture.
- Easy to develop within the semester.
- Reduced security concerns since payments are done externally.
- Better control of item availability through rental states.
- Improved accountability by limiting users to UB students.
- Direct communication between owners and customers.

Negatives

- Users must complete payment outside Remmogo, and the system cannot independently verify that money changed hands.
- Reservation and confirmation timeouts add lifecycle rules that must be tested carefully.
- Chat feature increases the development workload.

These decisions establish a layered, scope-controlled baseline for Phase 2 while preserving traceability to the approved requirements and quality scenarios.

9. Data, API/service and deployment design

Not applicable to the Phase 1 requirements-and-analysis baseline. Logical data contracts and deployment behaviour will be defined in Phase 2.

10. Detailed design, patterns and framework

Not applicable to the Phase 1 baseline. Detailed interfaces, patterns and framework boundaries will be selected after the architecture drivers are evaluated in Phase 2.

11. User-interface, accessibility and usability

Not applicable to the Phase 1 baseline. User-interface prototypes and accessibility evaluation are deferred to the detailed-design phase.

12. Implementation vertical slice

Not applicable to the Phase 1 baseline. The approved Request to Rent Item workflow is the intended vertical slice for later implementation.

13. Testing and results

No implementation test results are claimed in Phase 1. The acceptance criteria and quality scenarios define the tests to be executed in later phases.

14. Reconciliation and change history

The Lab 6 review identified three high-risk findings: inconsistent availability during reservation, unclear payment-system boundaries and lifecycle states that mixed interface actions with Rental states. The availability rule and consistency matrix were revised, payment processing was explicitly kept outside Remmogo, and the state machine was refocused on the Rental lifecycle. Related requirements, acceptance criteria, responsibilities, state descriptions and architecture decisions were reconciled. The diagrams embedded in this report are retained as supplied by the team.

15. Risks, technical debt and release recommendation

Residual risks include truthful external-payment confirmation, simultaneous requests for limited quantity, delayed return confirmations, misuse of chat and unresolved university/residence policy details. Phase 2 should prioritise transactional reservation logic, authorisation checks, auditable confirmation records and timeout/notification tests before expanding scope.

16. Academic integrity and AI-use record

AI assistance was used to review wording, consistency, traceability and document presentation. The team remains responsible for the project decisions, diagrams, verification of all content and the submitted repository baseline.

References

University of Botswana. CSI473 Laboratory 4, Laboratory 5 and Laboratory 6 student briefs.
Remmogo Project Problem Proposal and Approval Form, approved 17 August 2026.
Remmogo project repository: <https://github.com/LeeLeeGMOSPhidip/CSI473-Project>

Appendices

Appendix A contains the consistency matrix. The repository also retains the approval form, editable model sources, readable exports, CRC cards, business rules, acceptance criteria, quality scenarios, review checklist and architecture-driver note.

Appendix A Consistency Matrix

The matrix links use-case steps, sequence messages, domain responsibilities, lifecycle effects and requirement evidence for the Phase 1 baseline.

Use-case step	Proposed sequence message	CRC/domain responsibility	State or lifecycle effect	Requirement / rule / acceptance evidence
Sign in - 1. Student enters account credentials.	signIn(ubEmail, password)	Account authenticates the Student and protects account access.	Account must be active; no Rental state change.	FR-02; business rule: only registered users with valid UB accounts may access protected functions.
Search for items - 1. Customer enters an item name or category.	searchItems(query, category)	ItemListing provides searchable details; Category supports classification and filtering.	No Rental state change.	FR-05, FR-06; Quality Scenario 2.
View available items - 1. System returns matching available listings.	findAvailableListings(criteria)	ItemListing protects its availability status and quantity.	Guard: [availableQuantity > 0 AND status = AVAILABLE]; no Rental state change.	FR-07, FR-11; business rule: unavailable listings must not appear in the storefront.
View item details - 1. Customer selects a listing and views its details.	getItemDetails(listingId)	ItemListing supplies the item name, category, condition, description, rental price, and available quantity.	No Rental state change.	FR-04; Successful Rental Request acceptance criterion.
Send rental request - 1. Customer enters the rental period and quantity and submits the request.	sendRentalRequest(listingId, rentalPeriod, quantity)	Rental validates the request with Account and ItemListing; ItemListing verifies that the requested quantity is available.	Guard: [signed in AND valid UB account AND quantity available]; Rental enters PendingTransaction.	Use case: Send Rental Request; Successful Rental Request acceptance criterion; approved proposal workflow.
Send rental request - 2. System creates the pending Rental record.	createRental(customer, owner, listing, period, quantity)	Rental records the participants, listing, rental period, quantity, agreed price, and current status.	Rental: initial -> PendingTransaction.	Successful Rental Request acceptance criterion; Quality Scenario 3.
Send rental request - 3. System notifies the Owner of the request.	notifyOwner(rentalId)	Rental coordinates the notification to the correct Owner.	Rental remains PendingTransaction.	Successful Rental Request acceptance criterion.
Send rental request - A1. Requested quantity becomes unavailable or the request cannot be fulfilled.	cancelRental(reason)	Rental records the failed request; ItemListing prevents its available quantity from dropping below zero.	Rental: PendingTransaction -> Cancelled.	Prevent Duplicate Booking acceptance criterion; FR-07; business rule on item availability.
View rental request - 1. Owner views the pending request.	viewRentalRequest(rentalId)	Rental supplies the Customer, listing, rental period, requested quantity, agreed price, and status.	Rental remains PendingTransaction.	Use case: View Rental Request; approved proposal workflow.
View rental request - 2. Owner accepts the request while the requested quantity is available.	acceptRentalRequest(rentalId); reserveQuantity(quantity)	Rental records acceptance; ItemListing reserves the requested quantity to prevent double booking.	Guard: [quantity available]; Rental: PendingTransaction -> Reserved; the system then notifies the Customer and enters WaitingForCustomerConfirmation.	Proposal Objective 2; Prevent Duplicate Booking acceptance criterion; business rule on reserving available items.
View rental request - A1. Owner rejects the request or a participant cancels before activation.	cancelRental(reason); releaseQuantity(quantity)	Rental records cancellation; ItemListing releases any reserved quantity.	Rental: PendingTransaction/Reserved -> Cancelled.	Proposal Objectives 2 and 3; cancellation rule.
Chat with user or admin - 1. Customer and Owner communicate about rental arrangements.	sendMessage(rentalId, content)	Conversation contains each Message; only authorised participants may send and view messages.	No Rental state change.	FR-12; CRC communication responsibility; Quality Scenario 4.
Confirm payment made - 1. Customer confirms that payment was made outside Remmogo.	confirmPaymentMade(rentalId)	PaymentConfirmation records the confirming Customer and confirmation time; Rental prevents duplicate confirmation.	Rental: WaitingForCustomerConfirmation -> WaitingForOwnerConfirmation.	FR-08; Payment Confirmation acceptance criterion; explicit out-of-scope rule for payment processing.
Confirm payment received - 1. Owner confirms receiving the payment.	confirmPaymentReceived(rentalId)	A second PaymentConfirmation is recorded; Rental verifies that the Customer and Owner have both confirmed.	Guard: [customerConfirmed AND ownerConfirmed]; Rental remains WaitingForOwnerConfirmation until handover is confirmed.	FR-08; Payment Confirmation acceptance criterion; business rule requiring confirmation by both parties.

Use-case step	Proposed sequence message	CRC/domain responsibility	State or lifecycle effect	Requirement / rule / acceptance evidence
Payment confirmation - A1. The required confirmation is not received within 48 hours.	checkPaymentConfirmationDeadline(); cancelRental(reason); releaseQuantity(quantity); notifyParticipants()	Rental enforces the confirmation deadline and coordinates cancellation and notifications; ItemListing releases the reservation.	Rental: WaitingForCustomerConfirmation/WaitingForOwnerConfirmation -> Cancelled; Rental must not become ActiveRental.	Handling Payment Not Confirmed acceptance criterion; payment confirmation deadline rule.
Confirm item handover - 1. Owner confirms that the item has been handed over after both payment confirmations are present.	confirmHandover(rentalId)	Rental verifies both payment confirmations, records the handover and due date, and activates the rental; ItemListing retains the reduced available quantity.	Guard: [customerConfirmed AND ownerConfirmed]; Rental: WaitingForOwnerConfirmation -> ActiveRental.	Use case: Confirm Item Handover; approved proposal core workflow.
Overdue check - A1. The due date passes without a confirmed return.	checkDueDate(); markOverdue(); notifyParticipants()	Rental enforces the due date, marks the rental overdue, and coordinates notifications.	Rental: ActiveRental -> Overdue.	Return and Overdue Management acceptance criterion; proposal exception flow; Quality Scenario 6.
Confirm item return - 1. Customer records return confirmation and the item's condition.	confirmReturn(rentalId, condition)	ReturnConfirmation records the Customer, time, and condition; Rental prevents duplicate confirmation.	Rental remains ActiveRental or Overdue until both parties confirm the return.	FR-09; approved proposal core workflow.
Confirm item return - 2. Owner confirms receiving the returned item.	confirmReturn(rentalId, condition)	A second ReturnConfirmation is recorded; Rental verifies both confirmations and closes the transaction.	Guard: [customerConfirmed AND ownerConfirmed]; Rental: ActiveRental/Overdue -> Completed.	FR-09; Return and Overdue Management acceptance criterion; business rule on completed rentals.
Confirm item return - 3. System restores the returned quantity.	increaseAvailableQuantity(quantity)	ItemListing updates the available quantity without allowing it to exceed the total quantity.	Listing becomes visible whenever availableQuantity > 0; Rental remains Completed.	FR-07, FR-11; business rule on listing availability.
Rate user - 1. Either Rental participant rates the other participant.	submitRating(rentalId, score, comment)	Rating stores the feedback and associates it with the completed Rental and its two participating Students.	Guard: [Rental = Completed]; no Rental state change.	FR-10; business rule: only participants in a completed rental may rate each other.
Manage disputes - 1. Student reports a late, damaged, cancelled, or disputed rental.	submitReport(rentalId, type, description)	Report records the issue; Administrator reviews it and records its resolution.	Report: OPEN -> UNDER_REVIEW -> RESOLVED/DISMISSED; Rental state normally remains unchanged.	FR-13; Proposal Objective 3; business rule on administrative reports.